

## **Final Project Experiments**

### **Overview**

Four experiments characterise the fitness telemetry backend under realistic and adversarial load, run live against AWS ECS Fargate behind an Application Load Balancer provisioned via Terraform. The system under test: a Go/Gin HTTP server, in-memory time-series store (sync.RWMutex + sorted slices), sliding-window z-score anomaly detector (200-sample window), SSE broker (goroutine-per-client, non-blocking fan-out), and async ingest queue (10-worker pool, 1,024-slot ChannelQueue). Figures are collected in the appendix below.

### **Experiment 1 — Throughput Ceiling & Horizontal Scaling**

#### **Purpose**

Part A applies a step-load ramp (200-user increments, 200→400→600 users) against a single ECS task to find the single-task failure threshold. Using this, Part B fixes load at 600 users and compares 1, 2, and 4 tasks side by side. The core tradeoff is compute capacity vs. shared state: adding tasks adds goroutine scheduling capacity, but each task holds an independent in-memory store. ALB round-robin fragments the anomaly detector's rolling window across tasks, so detection quality degrades as the cluster grows.

#### **Results**

Part A: no failures through 400 users; failures appeared only at 600 users (see Figure 1). All failures were HTTP 504 Gateway Timeout exclusively on ingest endpoints — zero read failures. Ingest P50 was 98ms but P95 hit the ALB's 60-second idle-timeout ceiling at 60,000ms (see Figure 2).

From part B:

| Metric      | 1 Task      | 2 Tasks   | 4 Tasks |
|-------------|-------------|-----------|---------|
| Failures    | 413 (10.2%) | 0 (0%)    | 0 (0%)  |
| RPS         | 33.78       | 28.2      | 408.54  |
| Ingest P95  | 60,000ms    | ~15,000ms | 110ms   |
| Ingest P99  | 60,000ms    | ~45,000ms | 140ms   |
| CPU Peak    | 64.3%       | 33.8%     | 8.07%   |
| Memory Peak | 24.5%       | 21.7%     | 2.21%   |

#### **Analysis**

The bottleneck is RWMutex write-lock contention, not raw CPU exhaustion. At 64% CPU utilisation with 10.2% failures, a CPU-bound system would still be processing requests

slowly — not producing 504s. The explanation: 600 concurrent ingest goroutines all compete for the exclusive write lock on Store.Add. Goroutines blocked on a mutex do not burn CPU — they sit idle in the scheduler queue consuming goroutine stack memory and scheduler state without executing. They wait long enough for the ALB's 60-second idle timeout to fire, producing 504s before the server responds.

At 2 tasks, each task handles ~300 concurrent writers — below the 60-second failure threshold but still contention-heavy (P95 ~15,000ms). At 4 tasks, each task sees ~150 writers, the lock queue drains fast enough, and throughput recovers dramatically. This super-linear RPS jump is characteristic of lock-bound systems breaking free of a contention regime, not linear CPU scaling. 1 and 2 tasks were both stuck in saturation; 4 tasks crossed a threshold.

Memory remaining flat confirms the split-brain problem: each task only accumulates its fraction of total reads, so the z-score detector's 200-sample window fills at  $1/N$  the rate. The system scales throughput at the direct expense of detection accuracy. Replacing the in-memory RWMutex store with DynamoDB would convert lock acquisitions into network calls and centralise the rolling window across all tasks — the per-write latency (~1–5ms DynamoDB same-region) is far lower than the current lock-wait latency (up to 60,000ms under contention).

## **Experiment 2 — In-Memory Baseline Performance**

### **Purpose and Tradeoff**

This experiment characterises the in-memory store under controlled conditions across three operation types: 50 concurrent goroutine ingests (exposing write-lock contention), 50 sequential metric queries (exposing read performance), and stats aggregation against a pre-seeded 10,000-record store (exposing the  $O(n)$  scan cost). The tradeoff is simplicity vs. scalability: zero persistence overhead, but  $O(n)$  stats and RWMutex write serialisation impose ceilings that a database backend could raise.

### **Results**

| Operation           | P50     | P95     | P99     | Max     | n  |
|---------------------|---------|---------|---------|---------|----|
| Ingest (concurrent) | 1,037ms | 2,350ms | 3,990ms | 3,990ms | 50 |
| Query (sequential)  | 205ms   | 301ms   | 1,166ms | 1,166ms | 50 |
| Stats (10k records) | 214ms   | 292ms   | 364ms   | 364ms   | 50 |

All 150 operations succeeded. The 10,000-record store was seeded in 21.2 seconds via a 50-goroutine pool.

## Analysis

Concurrent ingest produced a staircase latency distribution: goroutines returned in distinct clusters (~790ms, ~881ms, ~1,037ms, up to ~3,990ms). This is RWMutex write-lock serialisation — goroutines competing for the same lock cycle return together; the outlier at 3,990ms waited through every preceding batch. This directly corroborates Experiment 1: the staircase is the same mechanism, just at 50 concurrent writers instead of 600. At 600, the staircase extends to 60 seconds and the ALB fires first.

Sequential reads were fast and stable (P50 205ms, one GC-pause outlier at 1,166ms). Shared read locks allow concurrent readers with no contention. Stats at 10,000 records (P50 214ms) showed no meaningful overhead over baseline query latency —  $O(n)$  scan cost is dominated by network round-trip at this scale. The ceiling exists but is not visible at 10k records; at 1,000,000 records the cost would be orders of magnitude higher.

## Experiment 3 — SSE Fan-Out Ceiling

### Purpose

This experiment finds the fan-out ceiling of the SSE broker under concurrent persistent connections while anomaly events are continuously generated. Three user classes apply simultaneous pressure: SSEClientUser holds /stream connections open for 20–40 seconds; AnomalyDriverUser injects extreme heart rate values (5, 8, 220, 225 bpm) at 4–8 req/s to drive z-score alerts into the stream; BulkheadProbeUser attempts to connect past the 200-client cap to verify 503 enforcement. Chaos injection via boto3 halves the ECS task count at the 4-minute mark to observe the reconnection storm (see Figure 10 for the pre-chaos two-task state).

The tradeoff is fan-out cost vs. connection capacity. Each anomaly broadcast iterates all connected client goroutines synchronously — CPU cost grows linearly with client count. The bulkhead caps this at 200 clients, trading connection capacity for CPU stability.

A secondary tradeoff emerged during implementation: Locust's gevent runtime does not hold iter\_content SSE connections reliably when no data is buffered, capping sustainable concurrent connections at ~85 per task regardless of user count. The bulkhead was lowered from 500 to 200 to produce a testable result; this harness limitation is documented as a measurement constraint, not a server-side ceiling.

### Results

| Metric                    | Value   |
|---------------------------|---------|
| Total requests            | 186,269 |
| SSE connections attempted | 47,176  |

|                                 |                |
|---------------------------------|----------------|
| Bulkhead fires (503 on /stream) | 38,486 (81.6%) |
| Bulkhead misses (200 on probe)  | 1,208          |
| Ingest P50 / P95                | 120ms / 180ms  |
| Peak concurrent SSE clients     | ~85 per task   |
| CPU peak                        | 99%            |
| Memory peak                     | 7%             |

## Analysis

The bulkhead enforces correctly. 38,486 503 responses on /stream confirm the 200-client cap fires reliably. This converts the silent ALB idle-timeout failures from the original Week 3 run (2.1% failure rate, connections hanging until the ALB killed them at 60s) into fast, explicit rejections — visible and recoverable by design.

CPU hitting 99% at only 85 concurrent clients is the central finding (see Figure 15). Each anomaly from AnomalyDriverUser triggers a broadcast that iterates all 85 connected client goroutines. At 4–8 anomalies/second the broadcast loop runs continuously, saturating the task's CPU allocation. The non-blocking select/default in sse.go prevents slow clients from stalling the broadcast path but does not reduce the  $O(n)$  iteration cost. Allowing 200 clients would double the per-broadcast cost, pushing the task into CPU saturation before the bulkhead fires — the ceiling is justified by compute cost.

Ingest was entirely unaffected by SSE pressure (P50 120ms, P95 180ms, zero failures), confirming the async queue design: ingest handlers enqueue and return without touching the SSE broker. ALB connection metrics during the test are shown in Figure 14.

Chaos injection at the 4-minute mark halved the task count from 2 to 1. Clients reconnected simultaneously to the surviving task, briefly pushing its connection count above 85. The bulkhead shed the excess, CPU spiked, and steady state recovered within ~30 seconds as the reconnection storm subsided.

## Experiment 4 — Queue Worker Saturation

### Purpose and Tradeoff

This experiment finds the queue saturation ceiling: the point at which the 10-worker goroutine pool cannot drain the 1,024-slot ChannelQueue fast enough and records are discarded. 180 zero-wait BurstIngestUsers each send 50-reading batches with no pause, alongside 20 DrainMonitorUsers polling /health every 2–3 seconds to track worker drain rate. This is maximum burst pressure — far beyond realistic Apple Health upload patterns.

The tradeoff is throughput vs. data integrity: the async design decouples HTTP

acknowledgement from processing, keeping responses fast under any load, but sacrifices records when workers fall behind. A secondary tradeoff emerged: the connection flood can overwhelm the HTTP accept queue before queue saturation even activates, producing ALB health check failures that are distinct from the application-level backpressure the experiment was designed to measure.

## Results

| Metric           | Value           |
|------------------|-----------------|
| Total requests   | 330,104         |
| Total failures   | 309,828 (93.9%) |
| Ingest RPS       | 1,093           |
| Ingest P50 / P95 | 94ms / 450ms    |
| /health 503s     | 1,302           |

Failure breakdown: 275,817 (84%) were HTTP 503 from the ALB; 31,579 were queue\_full drops; 6,708 were 502/504 task crash-restarts; 1,419 were /health drain stalls. CloudWatch confirmed the task was not resource-saturated in the traditional sense (see Figure 15) — CPU spiked during the flood, but the dominant failure mode was ALB-layer connection refusal.

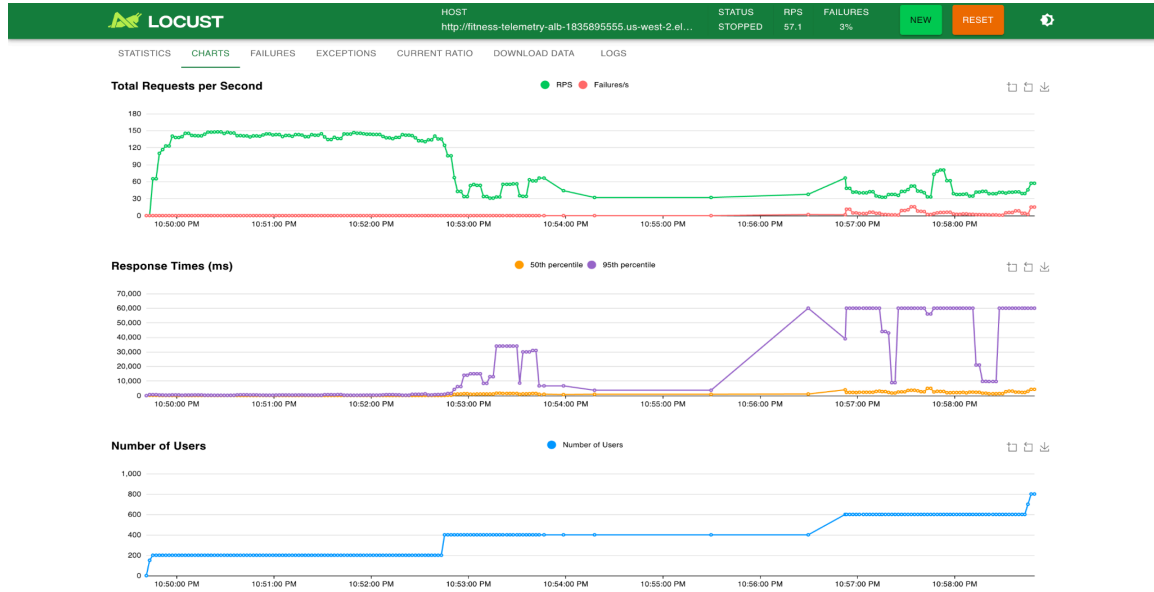
## Analysis

Two distinct failure regimes appeared. Regime 1 — queue saturation: at 1,093 RPS with 50-reading batches, the pipeline received ~54,650 readings/second. The 1,024-slot queue fills in under 20ms at that rate ( $1024 \div 54,650 \approx 18\text{ms}$ ). Ten workers processing Store.Add → Detector.Detect → Broker.Broadcast sequentially cannot drain anywhere near that rate.

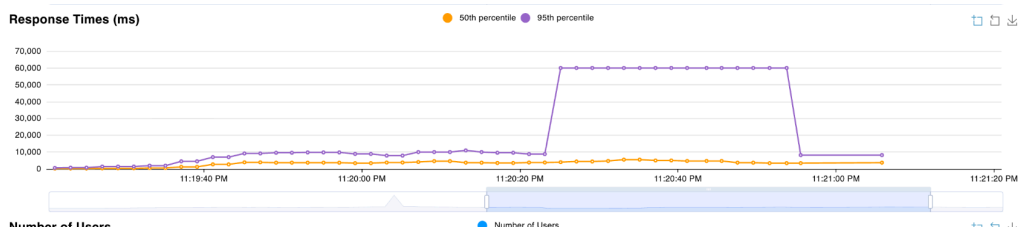
Regime 2 — HTTP layer collapse — is the more severe finding: 84% of failures were 503s from the ALB, meaning requests never reached the application. With 180 zero-wait users the Go HTTP server's goroutine pool exhausted and the ALB began refusing connections before ingest handlers could enqueue. The async queue's promise — "always return 202 fast" — broke down because the HTTP layer collapsed first. The 1,302 /health 503s confirm the task was completely unreachable, not just slow.

Requests that did get through were served quickly (P50 94ms, P95 450ms), confirming the async design works correctly when the HTTP layer is not overwhelmed. Increasing workerCount raises the queue ceiling but not the HTTP ceiling — the 503 storm requires more ECS tasks or higher Fargate vCPU allocation. The zero-wait load pattern is also unrealistically aggressive; real Apple Health exports are periodic batch uploads, not continuous zero-latency streams.

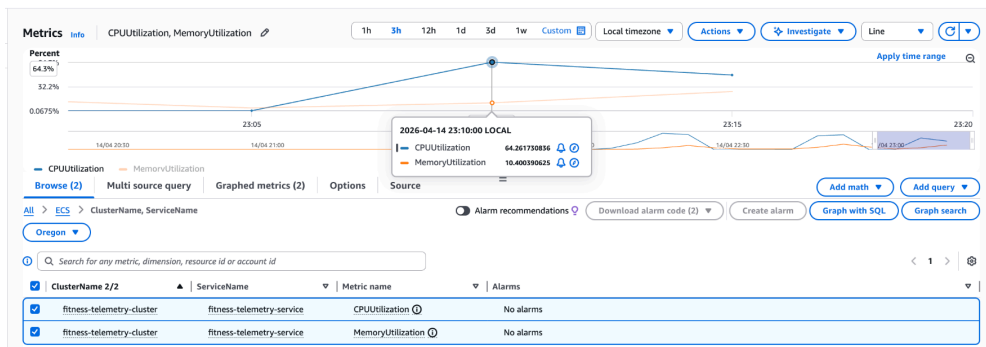
## Appendix of Figures



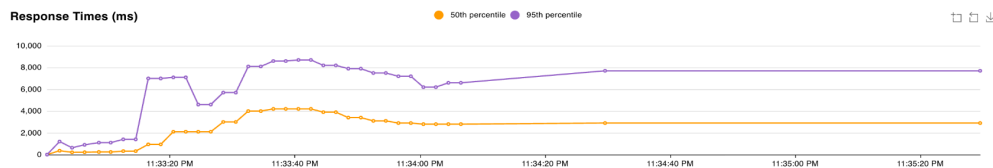
**Figure 1 — Locust step-load ramp — 1 task, 200→400→600 users; bimodal response time shape — Exp 1A**



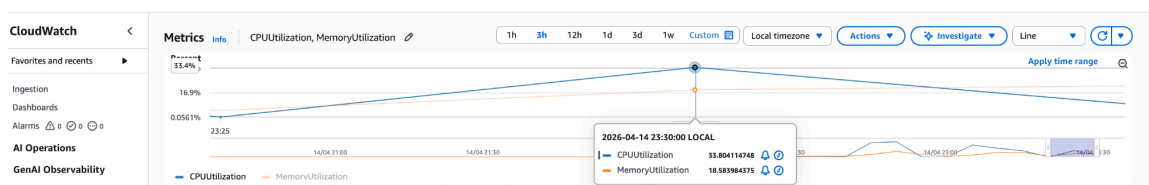
**Figure 2 — Locust response time chart — 1 task fixed at 600 users (P50 98ms / P95 60,000ms cliff) — Exp 1B**



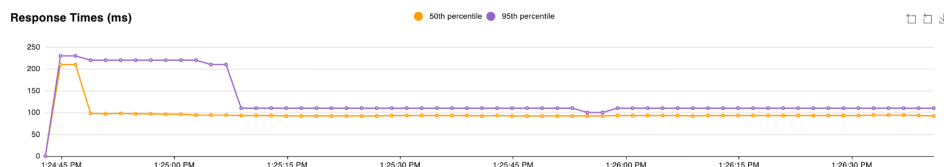
**Figure 3 — CloudWatch CPU/Memory — 1 task (CPU peak 64.3%, memory peak 24.5%) — Exp 1B**



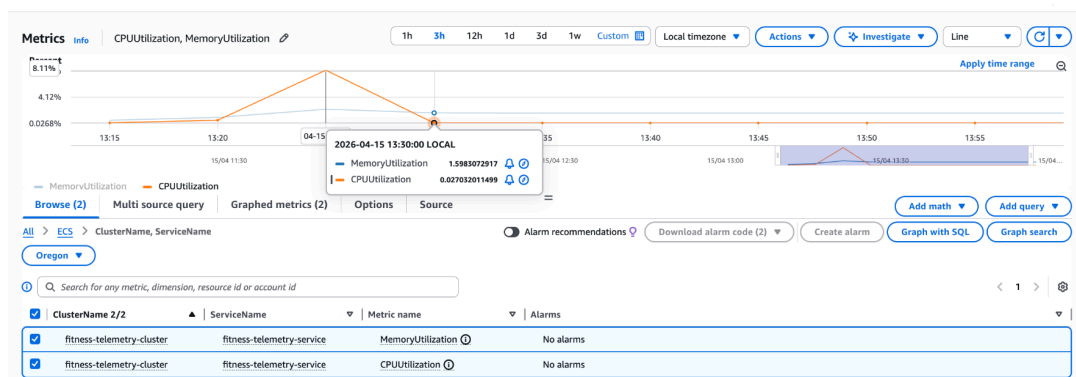
**Figure 4 — Locust response time chart — 2 tasks at 600 users (P95 drops, zero failures) — Exp 1B**



**Figure 5 — CloudWatch CPU/Memory — 2 tasks (CPU peak 33.8%, memory peak 21.7%) — Exp 1B**



**Figure 6 — Locust Response Times — 4 tasks; RPS jumps to 408 — Exp 1B**



**Figure 7 — CloudWatch CPU/Memory — 4-task run — Exp 1B**



Figure 8 — ALB CloudWatch RequestCount — Experiment 1 — Exp 1B

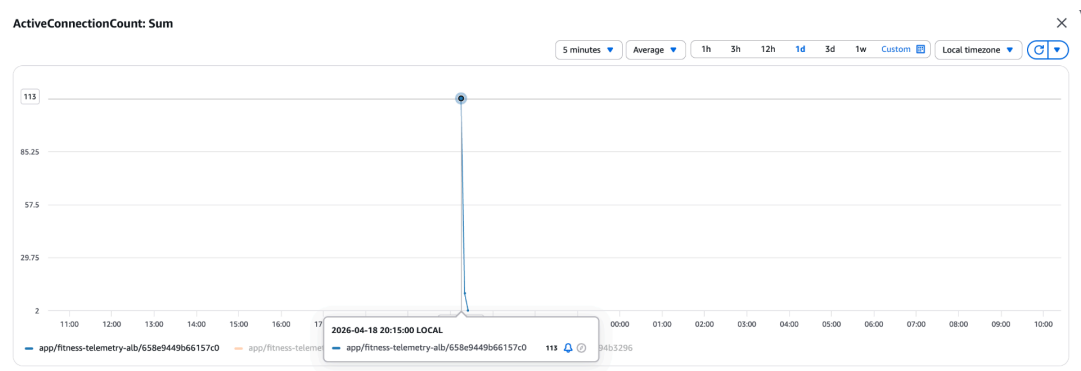


Figure 9 — ALB CloudWatch ActiveConnectionCount — Experiment 1 — Exp 1B

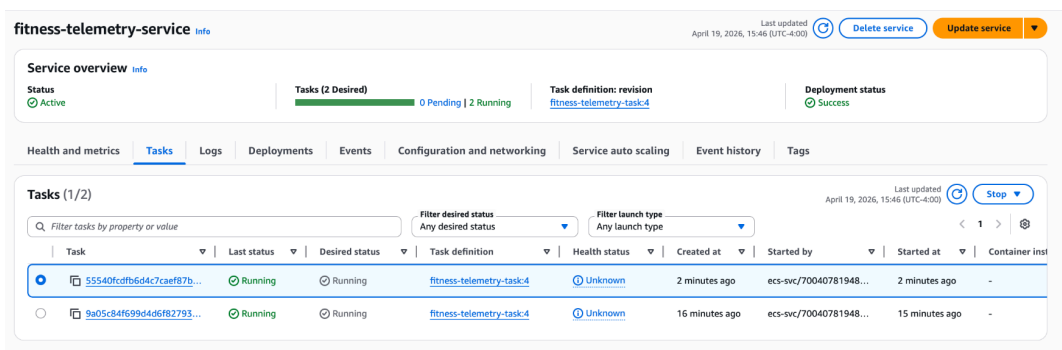


Figure 10 — ECS console — 2 tasks running before chaos injection — Exp 3

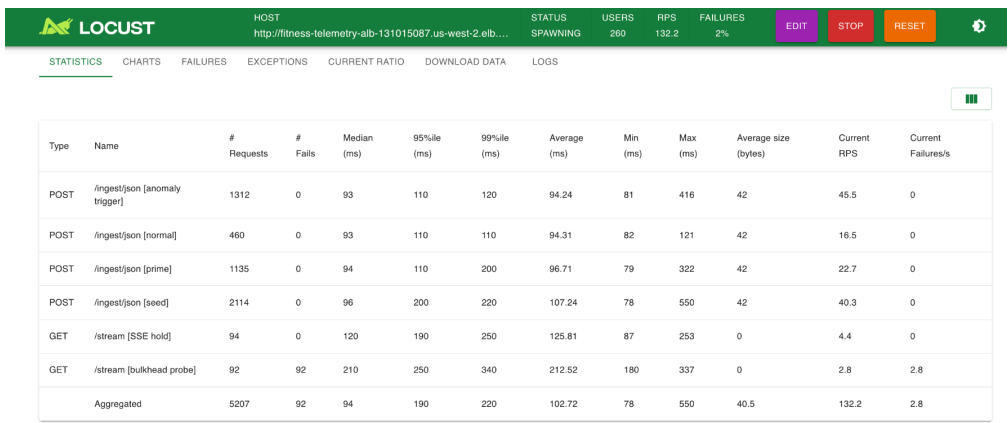


Figure 11 — Locust statistics at 260 users — bulkhead not yet at threshold — Exp 3



LOCUST

HOST

http://fitness-telemetry-alb-131015087.us-west-2.elb...

STATUS

SPAWNING

USERS

570

RPS

228.1

FAILURES

3%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

| Type       | Name                           | # Requests | # Fails | Median (ms) | 95%ile (ms) | 99%ile (ms) | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | Current RPS | Current Failures/s |
|------------|--------------------------------|------------|---------|-------------|-------------|-------------|--------------|----------|----------|----------------------|-------------|--------------------|
| POST       | /ingest/json [anomaly trigger] | 6435       | 0       | 100         | 170         | 200         | 112.05       | 80       | 448      | 42                   | 104.3       | 0                  |
| POST       | /ingest/json [normal]          | 2252       | 0       | 100         | 170         | 210         | 111.8        | 80       | 398      | 42                   | 34.8        | 0                  |
| POST       | /ingest/json [prime]           | 2525       | 0       | 99          | 170         | 230         | 107.01       | 78       | 355      | 42                   | 22.1        | 0                  |
| POST       | /ingest/json [seed]            | 4642       | 0       | 100         | 210         | 280         | 117.01       | 78       | 739      | 42                   | 40.6        | 0                  |
| GET        | /stream [SSE hold]             | 763        | 129     | 190         | 280         | 370         | 176.49       | 82       | 563      | 9.81                 | 19.7        | 8.9                |
| GET        | /stream [bulkhead probe]       | 413        | 373     | 230         | 300         | 380         | 236.94       | 179      | 481      | 5.62                 | 6.6         | 3.5                |
| Aggregated |                                | 17030      | 502     | 100         | 220         | 270         | 118.54       | 78       | 739      | 39.68                | 228.1       | 12.4               |

Figure 12 — Locust statistics at 570 users — bulkhead firing on GET /stream — Exp 3

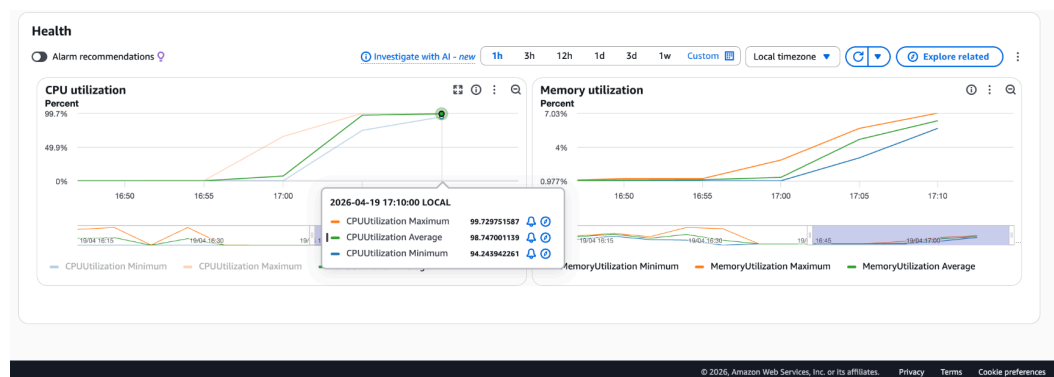


Figure 13 — ECS CloudWatch CPU and memory utilizations during SSE fan-out test — Exp 3

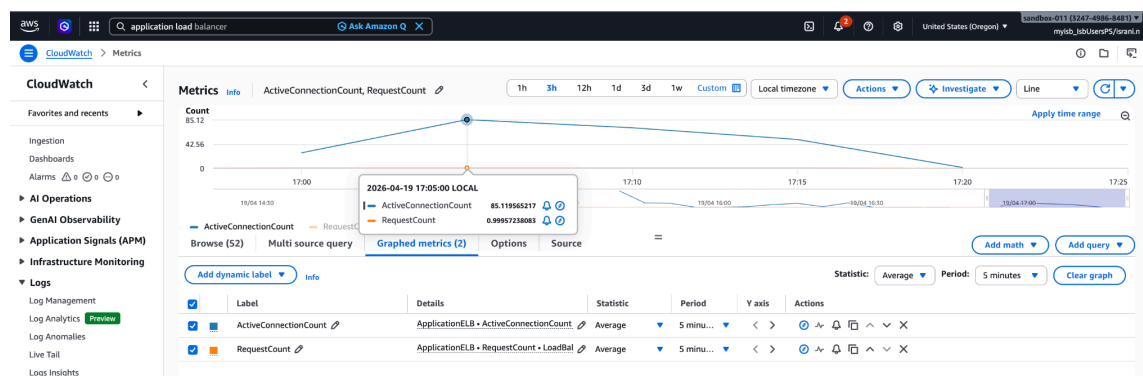


Figure 14 — ALB CloudWatch ActiveConnectionCount + RequestCount — Experiment 3 — Exp 3

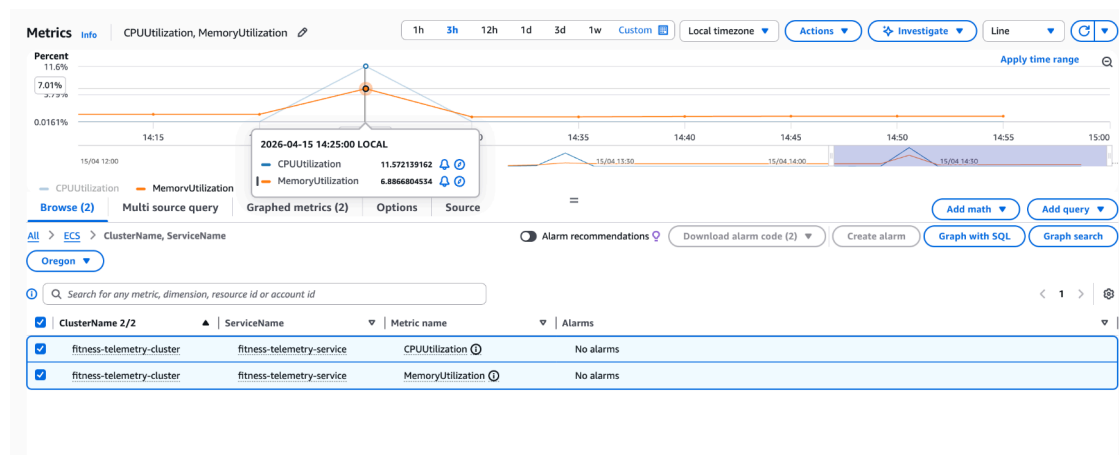


Figure 15 — CloudWatch CPU/Memory peak — Experiment 4 worker saturation — Exp 4